Personal AI Agent - Project Plan

Prepared for Dulaksha, October 2026. Status: planning

1. Vision

Build a personal AI agent that runs on my own computer: a customizable assistant that can reason, use tools (files, web, notes, shell), remember things across sessions, and talk to me through a working React app. Later, add a virtual narrator that speaks its replies.

Goals:

- Learn how agents really work by building the core loop myself.
- Have a portfolio-grade project to show for internships.
- Keep it customizable: persona, tools, memory, and models are all swappable.
- Keep it safe: sandboxed, with approval for risky actions.

Non-goals for v1: fine-tuning models, multi-user accounts, mobile app, cloud deployment.

2. Feasibility Summary

Verdict: feasible. A solid v1 is realistic in about 3-4 weeks of part-time work.

- Technical risk: low to medium. Every piece (LLM calls, tool calling, streaming UI, SQLite memory) is well documented.
- Main risk areas: local model quality and hardware limits, agent safety (shell and file access), scope creep.
- Cost: hosted API usage is small for personal use if capped; local models are free but need capable hardware.
- Skills used: Python or Node backend, React, REST/WebSocket streaming, databases, prompt engineering, security thinking.

3. Key Decisions

3.1 Hosted API vs Ollama (local)

	Hosted API	Ollama (local)
Reasoning and tool-use quality	Much better	Weaker, varies by model
Cost	Pay per token	Free after hardware
Privacy	Data leaves the machine	Fully local
Hardware needs	None	Decent GPU or lots of RAM
Offline use	No	Yes

Decision: support both behind one provider interface. Develop against a hosted API first, because it separates agent-logic bugs from weak-model bugs. Add Ollama for private and offline tasks. Ollama's OpenAI-compatible endpoint, or a library like LiteLLM, keeps this simple.

Rough local model guide: 7-8B models run on about 8 GB VRAM or 16 GB RAM but are mediocre agents; 14B+ models are much better and need more hardware.

3.2 Framework vs hand-written loop

Decision: write the core agent loop by hand for v1 (about 100 lines): call the model, run the requested tool, feed the result back, repeat until done. Reasons: it is the part most worth understanding, frameworks change often, and the needs are simple. Consider LangGraph later if workflows become complex (branching, retries, human-approval pauses, checkpointing).

3.3 Backend language

Python with FastAPI is the recommended default (AI ecosystem, easy streaming). Node with Express is a fine alternative. Still to decide.

4. Architecture

```
React UI (Vite)  <-- WebSocket / SSE -->  Backend (FastAPI)
                                           |- Agent loop
                                           |- LLM provider layer (API |
Ollama)
                                           |- Tools (files, shell, web, notes
...)
                                           |- Memory (SQLite + optional
vector store)
                                           |- Permission / approval layer
                                           '- (later) Voice layer: TTS / STT
```

Principles:

- Each layer sits behind its own interface so parts can be swapped (LLM provider, TTS engine, memory store).
- Replies stream in chunks end to end.
- Tool calls are structured data, shown separately from reply text in the UI.

5. How Conversation History Works

Model APIs are stateless: every request must include the whole conversation so far. The app therefore stores messages in a database, loads them for the active conversation, appends the new message, and sends the full list each time. Tool calls and tool results are also messages in that list.

Data model (SQLite):

- conversations (id, title, created_at)
- messages (id, conversation_id, role, content, tool_calls, created_at)
- settings (provider, model, persona, limits)
- memories (id, text, created_at) for long-term notes

Context window management (added as needed):

- Truncate oldest messages.
- Summarize older parts and keep the summary plus recent messages.
- Trim large old tool outputs.
- Use prompt caching on the stable beginning of the conversation.

- Retrieve old information from a vector store only when relevant.

Cross-conversation memory: save facts or notes in the database and inject relevant ones into the system prompt of new conversations.

6. Feature List

Core (v1)

- Streaming chat UI in React with conversation list and history
- Provider layer with a hosted API provider and an Ollama provider, switchable in settings
- Agent loop with tool calling and a maximum step limit
- Initial tools: sandboxed file read/write, web search, notes/todo store, shell command runner (approval required)
- Visible tool-call steps in the UI (for example, Reading file X)
- Approve / deny prompts for risky actions
- Editable persona and system prompt
- Per-conversation storage in SQLite
- Spending cap and token usage display

Version 2

- Long-term memory (notes injected into new chats; later embeddings with Chroma or sqlite-vec)
- Context management (summarization, trimming, caching)
- Saved skills / workflows: reusable instructions for repeatable jobs, such as summarizing lecture notes in my format
- Model routing: cheap or local model for easy tasks, stronger model for hard ones
- MCP support: build at least one tool as an MCP server
- Document search (RAG) over chosen folders
- Export and search of past conversations

Version 3 and beyond

- Virtual narrator (see section 7)
- Voice input with local speech-to-text (Whisper / faster-whisper)

- Desktop wrapper (Tauri or Electron) with system tray and global shortcut
- Scheduled and background tasks
- Optional framework migration (LangGraph) for complex workflows
- Extra tools: calendar, email drafts, code runner, screen or clipboard helpers

7. Virtual Narrator (planned for later)

A narrator is text-to-speech attached to the agent's output. The agent loop does not change; a voice layer takes reply text, produces audio, and optionally drives an avatar.

```
Agent reply text -> clean text -> TTS engine -> audio stream -> React plays
it
                                                        '-> (optional) avatar lip-
sync
```

TTS options:

- Browser Web Speech API: free, basic quality, ideal for a first prototype
- Piper: free, local, good quality, fast on CPU
- Kokoro: free, local, very natural for its size
- Coqui XTTS: local, very good, can clone voices, prefers a GPU
- ElevenLabs, OpenAI TTS and similar: best quality, pay per character

Making it feel like a narrator:

- Stream speech sentence by sentence so audio starts within a second or two.
- Strip markdown, code blocks, and URLs before speaking; show code on screen only.
- Allow interruption (stop button, or stop when the user starts speaking).
- Give it a persona through voice choice and a tone-setting system prompt.

Avatar levels:

1. Audio only, with an orb or waveform in the UI
2. 2D avatar with simple volume-based mouth movement
3. 3D avatar (Three.js / react-three-fiber with a VRM model) using viseme lip-sync

Prepare now: stream replies in chunks, put TTS behind its own interface, and keep reply text separate from tool-call displays.

8. Customization Levers

1. System prompt and persona (cheapest, covers most of the personality)
2. Tools (define what the agent can do)
3. Memory (what it knows about me and my projects)
4. Skills / workflows (saved instructions for repeatable tasks)
5. Model routing (which model for which task)
6. Fine-tuning (deliberately skipped unless prompting, tools, and memory prove insufficient)

9. Tech Stack

Layer	Choice
Frontend	React + Vite, streaming via WebSocket or SSE
Backend	FastAPI (Python), Node as an alternative
LLM access	Provider SDK or LiteLLM; Ollama for local models
Storage	SQLite; Chroma or sqlite-vec later
Tool standard	MCP (at least one tool built as an MCP server)
Voice (later)	Web Speech API, then Piper or Kokoro; Whisper for speech-to-text
Desktop (later)	Tauri or Electron

10. Security and Safety

- Sandbox: the agent works inside a dedicated folder; no blanket access to the whole machine.
- Approvals: shell commands, deletions, and anything outside the sandbox require explicit approval in the UI.
- Prompt injection: web pages and files can contain text that tries to hijack the agent. Treat all tool output as untrusted data, never as instructions, and do not auto-run actions requested by fetched content.
- Secrets: API keys in environment variables, never in the repo or in prompts.

- Cost control: spending cap on the API key and a maximum number of agent steps per request.
- Logging: keep a log of every tool call for review.

11. Roadmap

Week 1 - Foundation

- Project setup: React + Vite frontend, backend, repository
- Provider interface with one hosted API provider
- Streaming chat endpoint and a basic chat UI
- SQLite conversations and messages

Week 2 - The agent

- Hand-written agent loop with tool calling and a step limit
- First 2-3 tools (files, notes, web search)
- Tool calls displayed in the UI

Week 3 - Safety and memory

- Approval layer and sandbox for risky tools
- Shell tool with approval
- Conversation history polish, settings page (persona, provider, model)
- Spending cap and usage display

Week 4 - Local models

- Ollama provider and model switching
- Test which local models handle tool calling reliably
- Basic long-term memory notes

Week 5 onward - Extend

- Context management, skills/workflows, MCP, RAG
- Voice: TTS narrator, then speech input
- Desktop wrapper and avatar

Milestone checks: end of week 2, the agent completes a multi-step task using tools; end of week 4, the same agent runs on a local model; end of week 6, a demo-ready version with write-up.

12. Risks and Mitigations

- Weak local models: keep the hosted API as the default, and test local models per task.
- Hardware limits: choose model size to fit available GPU/RAM; avoid running local LLM and local TTS at full size at once.
- Unsafe actions: sandbox, approvals, logging.
- Scope creep: finish each week's milestone before adding extras; v1 stays small.
- API cost surprises: hard cap, step limits, trimming of long tool output.
- Context growth: summarization and trimming from v2.

13. Open Decisions

- Computer specs (GPU, VRAM, RAM), which determine realistic local models
- Backend language: Python/FastAPI or Node
- Which hosted API to start with
- Project name and repository structure
- Whether the narrator should be audio only or an avatar

14. Portfolio and Interview Value

What this project demonstrates: understanding of the agent loop and tool calling, provider abstraction, streaming UIs, state and memory design, security thinking (sandboxing and prompt injection), and the ability to evaluate local versus hosted models. Document the architecture, include a short demo video, and write up what was learned about model trade-offs.